

Виртуальное окружение Python

Что это такое:

- Это изолированная «песочница» для Python-проекта.
- В ней можно устанавливать нужные библиотеки без риска испортить глобальный Python.

Зачем:

- Разные проекты могут требовать разные версии библиотек.
- Можно легко «обновлять» зависимости для одного проекта, не трогая другие.
- Удобно для автоматизации тестирования и деплоя.

1. Создание виртуального окружения

На Windows:

```
python -m venv venv
```

- **venv** — имя папки для окружения (можно любое)

На Mac / Linux:

```
python3 -m venv venv
```

2. Активация виртуального окружения

Windows (cmd):

```
venv\Scripts\activate
```

Windows (PowerShell):

```
venv\Scripts\Activate.ps1
```

Mac / Linux:

```
source venv/bin/activate
```

Проверка

```
(venv) C:\project>
```

- Префикс **(venv)** показывает, что окружение активно

3. Установка библиотек внутри окружения

```
pip install requests
```

- **requests** установится только в этом окружении, не тронув глобальный Python.

4. Деактивация окружения

```
deactivate
```

- Возвращает вас к глобальному Python.

Аналоги виртуального окружения в Python

1. **virtualenv**

- Похож на venv, но более старый и независимый от версии Python.
- Можно использовать для проектов с разными версиями Python.
- Пример установки:

```
pip install virtualenv  
virtualenv my_env
```

2. **conda** (Anaconda / Miniconda)

- Универсальный менеджер пакетов и окружений, особенно для Data Science.
- Создание окружения:

```
conda create -n my_env python=3.13  
conda activate my_env
```

3. Poetry

- Современный инструмент для управления зависимостями и окружениями.
- Автоматически создаёт виртуальное окружение для проекта.
- Пример:

```
poetry new my_project  
cd my_project  
poetry install  
poetry shell
```

4. Pipenv

- Интегрирует управление зависимостями и виртуальными окружениями.
- Удобен для простых проектов и автоматического создания Pipfile.
- Пример:

```
pip install pipenv  
pipenv install requests  
pipenv shell
```